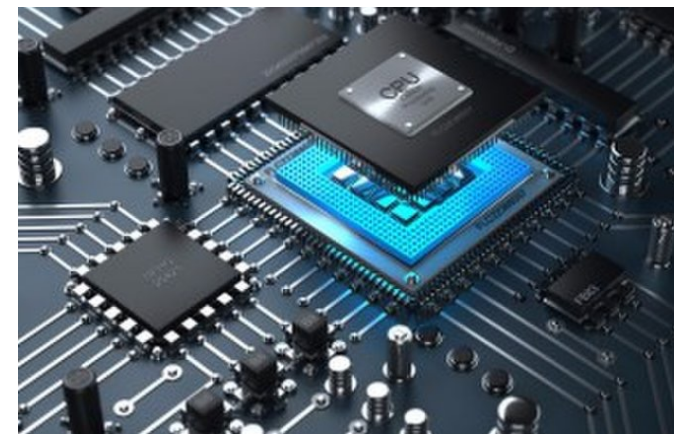


```
loop: lw    $t3, 0($t0)
      lw    $t4, 4($t0)
      add   $t2, $t3, $t4
      sw    $t2, 8($t0)
      addi  $t0, $t0, 4
      addi  $t1, $t1, -1
      bgtz  $t1, loop
```

Assembler

```
0x8d0b0000
0x8d0c0004
0x016c5020
0xad0a0008
0x21080004
0x2129ffff
0x1d20fff9
```



## Lecture 12:

# Recursive Functions in MIPS Assembly Intro to Digital Logic Design (Symbols)

*CMPSC 64, SPRING 2026*

*ZIAD MATNI, PH.D.*

*UNIVERSITY OF CALIFORNIA, SANTA BARBARA*

# Administrative

---

New homework due Monday!!

New readings for this week!

- **Handout 5** on Functions and the MIPS Calling Convention

*No Worksheet today...* 😞

**Reminder: Office Hours R 4 U!!!!**

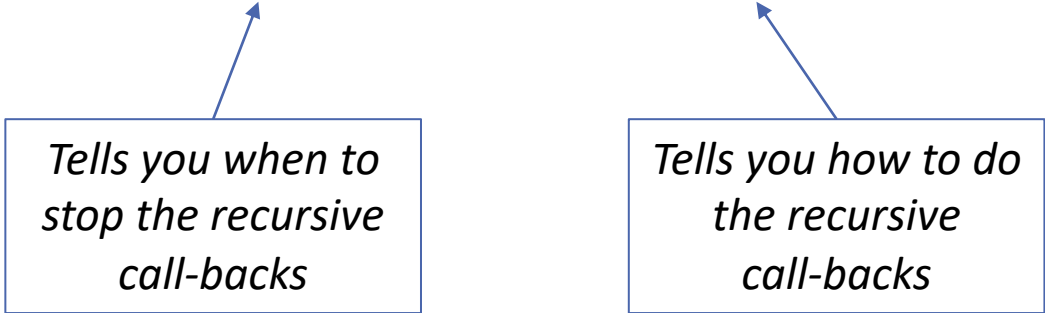
# An Example of Recursion

---

How can I make a recursive function for  
“**add all the integers from 1 to N**” ??

In the definition of the function, CALL the function itself!

- Remember: you need a **base-case** and a **recursion** case



*Tells you when to  
stop the recursive  
call-backs*

*Tells you how to do  
the recursive  
call-backs*

# An Example of Recursion

```
int addInts(int n) {  
    if (n <= 0)  
        return 0; // base case  
    return ( n + addInts(n-1) ); // recursive case  
}  
  
int main() {  
    int x = 3;  
    cout << addInts(x);  
    return 0;  
}
```

Do we need to preserve **return address(es)**?  
Why/why not?

Do we need to preserve the variable **n**?  
Why/why not?

I expect the following to happen:  
  
The recursion does: 3 + 2 + 1 + 0,  
  
so, the program prints out **6**

What regs will we have to preserve??

ANS:  
The variable **n**  
(i.e. register **\$a0**)  
  
AND the register **\$ra** (why?)

```
.text
addInts:
# $a0: n
# $v0: result
# addInts(n) = sum of all numbers from 0 to n
    addiu $sp, $sp, -8      # PUSH $ra and $s0 into stack
    sw $ra, 4($sp)
    sw $s0, 0($sp)

    move $s0, $a0          # preserve a0 (variable n)
    li $v0, 0              # set the base case to 0
    ble $a0, $zero, return # If size <= 0, then you've reached the base case

    addi $a0, $a0, -1      # n is now: n - 1
    jal addInts            # recursive call

return:
    add $v0, $v0, $s0      # add n to $v0

    lw $ra, 4($sp)         # POP $ra and $sa back from stack
    lw $s0, 0($sp)
    addiu $sp, $sp, 8

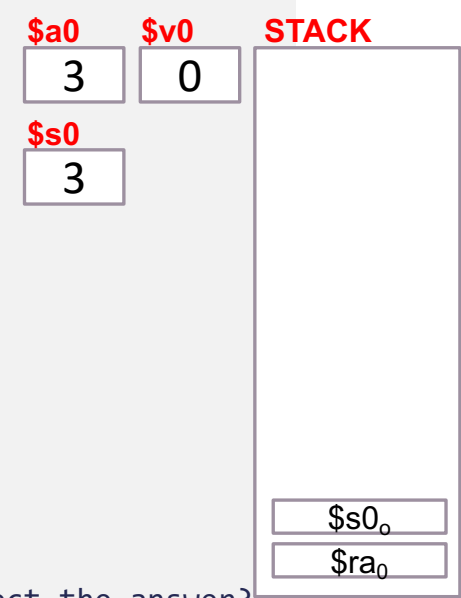
    jr $ra

main:
    li $a0, 3              # n = 3 (arbitrary, for this example)
    jal addInts            # call function for arg = 3, where do I expect the answer?

exit:
    move $a0, $v0          # print the answer
    li $v0, 1
    syscall

    li $v0, 10             # exit program
    syscall
```

```
int addInts(int n) {
    if (n <= 0)
        return 0;           // base case
    return (n + addInts(n-1)); // recursive case
}
```



What regs will we have to preserve??

ANS:

The variable **n**  
(i.e. register **\$a0**)

```
.text
addInts:
# $a0: n
# $v0: result
# addInts(n) = sum of all numbers from 0 to n
    addiu $sp, $sp, -8    # PUSH $ra and $s0 into stack
    sw $ra, 4($sp)
    sw $s0, 0($sp)
```

```
    move $s0, $a0        # preserve a0 (variable n)
    li $v0, 0            # set the base case to 0
    ble $a0, $zero, return # If size <= 0, then you've reached the base case
```

```
    addi $a0, $a0, -1    # n is now: n - 1
    jal addInts          # recursive call
```

**return:**

```
$ra1 → add $v0, $v0, $s0    # add n to $v0

    lw $ra, 4($sp)        # POP $ra and $sa back from stack
    lw $s0, 0($sp)
    addiu $sp, $sp, 8

    jr $ra
```

**main:**

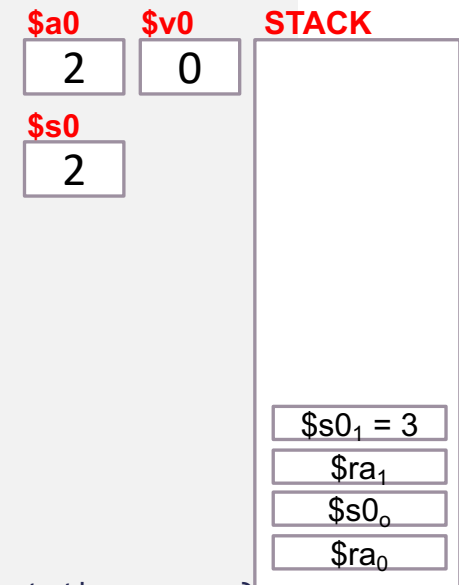
```
    li $a0, 3            # n = 3 (arbitrary, for this example)
    jal addInts          # call function for arg = 5, where do I expect the answer??
```

**exit:**

```
$ra0 → move $a0, $v0        # print the answer
    li $v0, 1
    syscall

    li $v0, 10          # exit program
    syscall
```

```
int addInts(int n) {
    if (n <= 0)
        return 0;           // base case
    return (n + addInts(n-1)); // recursive case
}
```





```

.text
addInts:
# $a0: n
# $v0: result
# addInts(n) = sum of all numbers from 0 to n
    addiu $sp, $sp, -8      # PUSH $ra and $s0 into stack
    sw $ra, 4($sp)
    sw $s0, 0($sp)

```

```

    move $s0, $a0          # preserve a0 (variable n)
    li $v0, 0              # set the base case to 0

```

```

    ble $a0, $zero, return # If size <= 0, then you've reached the base case

```

```

    addi $a0, $a0, -1      # n is now: n - 1
    jal addInts            # recursive call

```

```

return:
$ra1,2,3 → add $v0, $v0, $s0 # add n to $v0

```

```

    lw $ra, 4($sp)        # POP $ra and $sa back from stack
    lw $s0, 0($sp)
    addiu $sp, $sp, 8

    jr $ra

```

```

main:
    li $a0, 3             # n = 3 (arbitrary, for this example)
    jal addInts           # call function for arg = 3, where do I expect the answer?

```

```

exit:
$ra0 → move $a0, $v0      # print the answer
    li $v0, 1
    syscall

```

```

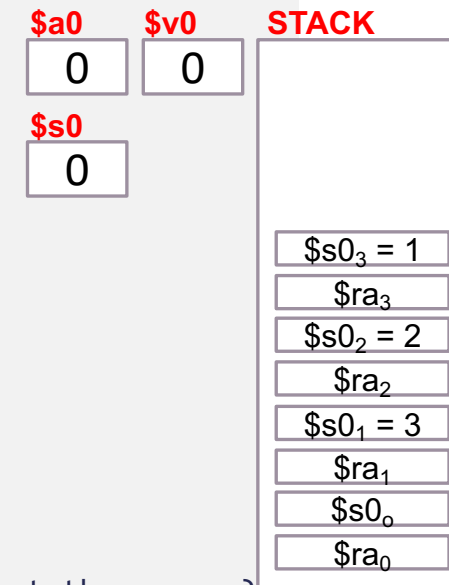
    li $v0, 10           # exit program
    syscall

```

```

int addInts(int n) {
    if (n <= 0)
        return 0;           // base case
    return (n + addInts(n-1)); // recursive case
}

```





```

.text
addInts:
# $a0: n
# $v0: result
# addInts(n) = sum of all numbers from 0 to n
    addiu $sp, $sp, -8      # PUSH $ra and $s0 into stack
    sw $ra, 4($sp)
    sw $s0, 0($sp)

    move $s0, $a0          # preserve a0 (variable n)
    li $v0, 0              # set the base case to 0
    ble $a0, $zero, return # If size <= 0, then you've reached the base case

    addi $a0, $a0, -1      # n is now: n - 1
    jal addInts            # recursive call

```

**return:**

```

$ra1,2,3 → add $v0, $v0, $s0 # add n to $v0

    lw $ra, 4($sp)        # POP $ra and $sa back from stack
    lw $s0, 0($sp)
    addiu $sp, $sp, 8

    jr $ra

```

**main:**

```

    li $a0, 3             # n = 3 (arbitrary, for this example)
    jal addInts           # call function for arg = 3, where do I expect the answer?

```

**exit:**

```

$ra0 → move $a0, $v0     # print the answer
    li $v0, 1
    syscall

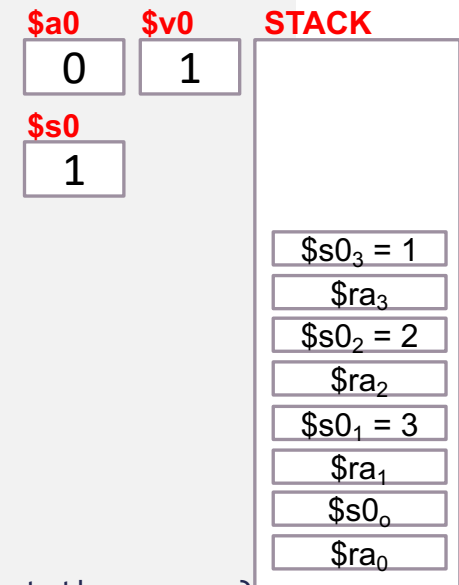
    li $v0, 10           # exit program
    syscall

```

```

int addInts(int n) {
    if (n <= 0)
        return 0;          // base case
    return (n + addInts(n-1)); // recursive case
}

```



```

.text
addInts:
# $a0: n
# $v0: result
# addInts(n) = sum of all numbers from 0 to n
    addiu $sp, $sp, -8      # PUSH $ra and $s0 into stack
    sw $ra, 4($sp)
    sw $s0, 0($sp)

    move $s0, $a0          # preserve a0 (variable n)
    li $v0, 0              # set the base case to 0
    ble $a0, $zero, return # If size <= 0, then you've reached the base case

    addi $a0, $a0, -1      # n is now: n - 1
    jal addInts            # recursive call

```

**return:**

```

$ra1,2 → add $v0, $v0, $s0 # add n to $v0

    lw $ra, 4($sp)        # POP $ra and $sa back from stack
    lw $s0, 0($sp)
    addiu $sp, $sp, 8

    jr $ra

```

**main:**

```

    li $a0, 3             # n = 3 (arbitrary, for this example)
    jal addInts           # call function for arg = 3, where do I expect the answer?

```

**exit:**

```

$ra0 → move $a0, $v0     # print the answer
    li $v0, 1
    syscall

    li $v0, 10            # exit program
    syscall

```

```

int addInts(int n) {
    if (n <= 0)
        return 0;          // base case
    return (n + addInts(n-1)); // recursive case
}

```

| \$a0 | \$v0 | STACK |
|------|------|-------|
| 0    | 3    |       |
| \$s0 |      |       |
| 2    |      |       |

|                       |
|-----------------------|
| \$s0 <sub>2</sub> = 2 |
| \$ra <sub>2</sub>     |
| \$s0 <sub>1</sub> = 3 |
| \$ra <sub>1</sub>     |
| \$s0 <sub>0</sub>     |
| \$ra <sub>0</sub>     |

```

.text
addInts:
# $a0: n
# $v0: result
# addInts(n) = sum of all numbers from 0 to n
    addiu $sp, $sp, -8      # PUSH $ra and $s0 into stack
    sw $ra, 4($sp)
    sw $s0, 0($sp)

    move $s0, $a0          # preserve a0 (variable n)
    li $v0, 0              # set the base case to 0
    ble $a0, $zero, return # If size <= 0, then you've reached the base case

    addi $a0, $a0, -1      # n is now: n - 1
    jal addInts            # recursive call

```

**return:**

```

$ra1 → add $v0, $v0, $s0 # add n to $v0

    lw $ra, 4($sp)        # POP $ra and $sa back from stack
    lw $s0, 0($sp)
    addiu $sp, $sp, 8

    jr $ra

```

**main:**

```

    li $a0, 3             # n = 3 (arbitrary, for this example)
    jal addInts           # call function for arg = 3, where do I expect the answer?

```

**exit:**

```

$ra0 → move $a0, $v0    # print the answer
    li $v0, 1
    syscall

    li $v0, 10            # exit program
    syscall

```

```

int addInts(int n) {
    if (n <= 0)
        return 0;          // base case
    return (n + addInts(n-1)); // recursive case
}

```

**\$a0** **\$v0** **STACK**

|             |                 |  |
|-------------|-----------------|--|
| 0           | 6               |  |
| <b>\$s0</b> | s0 <sub>0</sub> |  |

|                       |
|-----------------------|
| \$s0 <sub>1</sub> = 3 |
| \$ra <sub>1</sub>     |
| \$s0 <sub>0</sub>     |
| \$ra <sub>0</sub>     |

```

.text
addInts:
# $a0: n
# $v0: result
# addInts(n) = sum of all numbers from 0 to n
    addiu $sp, $sp, -8      # PUSH $ra and $s0 into stack
    sw $ra, 4($sp)
    sw $s0, 0($sp)

    move $s0, $a0          # preserve a0 (variable n)
    li $v0, 0              # set the base case to 0
    ble $a0, $zero, return # If size <= 0, then you've reached the base case

    addi $a0, $a0, -1      # n is now: n - 1
    jal addInts            # recursive call

return:
    add $v0, $v0, $s0      # add n to $v0

    lw $ra, 4($sp)         # POP $ra and $sa back from stack
    lw $s0, 0($sp)
    addiu $sp, $sp, 8

    jr $ra

main:
    li $a0, 3              # n = 3 (arbitrary, for this example)
    jal addInts            # call function for arg = 3, where do I expect the answer?

exit:
    move $a0, $v0          # print the answer
    li $v0, 1
    syscall

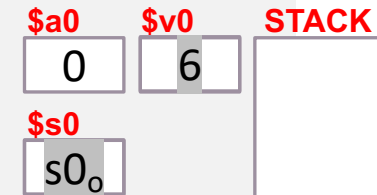
    li $v0, 10             # exit program
    syscall

```

```

int addInts(int n) {
    if (n <= 0)
        return 0;           // base case
    return (n + addInts(n-1)); // recursive case
}

```



**\$ra<sub>0</sub>** → move \$a0, \$v0 # print the answer

```

li $v0, 1
syscall

```

# Try This Exercise Yourself!

---

Translate this C++ program into MIPS assembly and implement the MIPS Calling Convention:

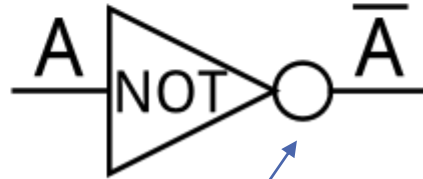
```
int rfunc (int a, int b) {  
    if (a <= b) { return 1; }  
    return 1 + rfunc(a - 2, b);  
}  
  
int main () {  
    int x(5), y(1);  
    cout << rfunc(x, y);  
    return 0;  
}
```

# An Introduction to the Symbols Used in Digital Logic Design!

# Graphical Symbols and Truth Tables

## *NOT*

*Input on the left*



*Output on the right*

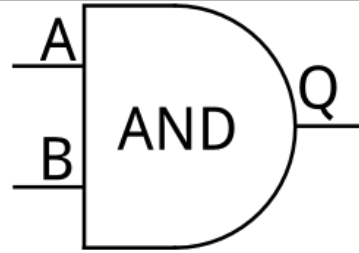
| A | $\overline{A}$ | !A |
|---|----------------|----|
| 0 |                | 1  |
| 1 |                | 0  |

*The bubble means **inversion** (i.e. **opposite**) of the bit!!*

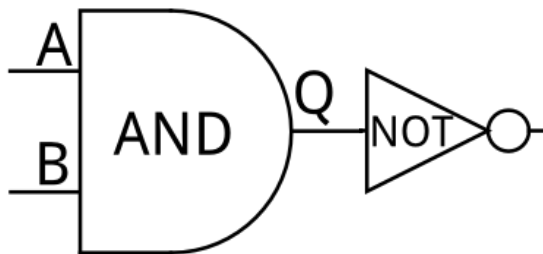
# Graphical Symbols and Truth Tables

## *AND and NAND*

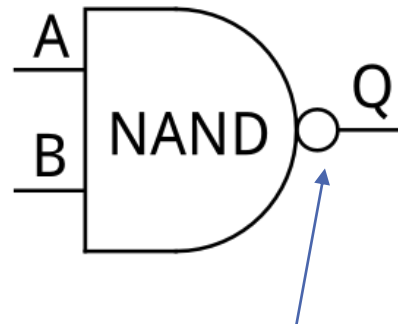
**Practice Drawing  
this Symbol!**



| A | B | A . B |
|---|---|-------|
| 0 | 0 | 0     |
| 0 | 1 | 0     |
| 1 | 0 | 0     |
| 1 | 1 | 1     |



≡



*There's that bubble again!*

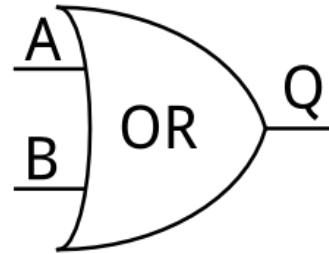
| A | B | $\overline{A . B}$<br>!(A.B) |
|---|---|------------------------------|
| 0 | 0 | 1                            |
| 0 | 1 | 1                            |
| 1 | 0 | 1                            |
| 1 | 1 | 0                            |



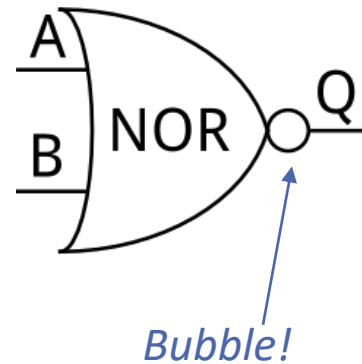
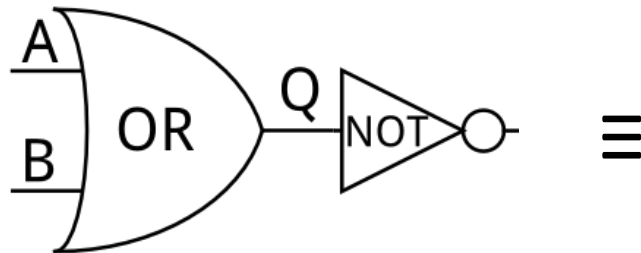
# Graphical Symbols and Truth Tables

## *OR and NOR*

**Practice Drawing  
this Symbol!**



| A | B | A + B |
|---|---|-------|
| 0 | 0 | 0     |
| 0 | 1 | 1     |
| 1 | 0 | 1     |
| 1 | 1 | 1     |

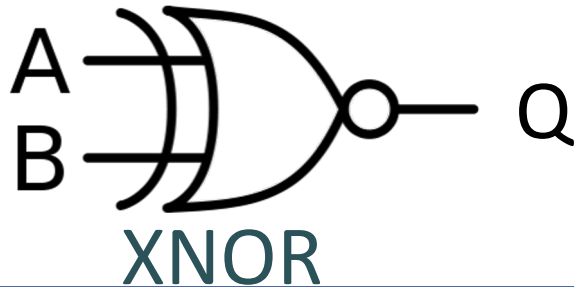
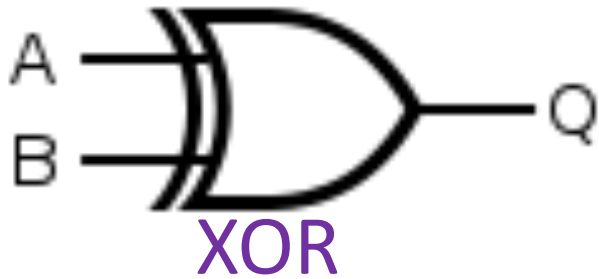


| A | B | $\overline{A + B}$<br>!(A + B) |
|---|---|--------------------------------|
| 0 | 0 | 1                              |
| 0 | 1 | 0                              |
| 1 | 0 | 0                              |
| 1 | 1 | 0                              |

# Graphical Symbols and Truth Tables

## *XOR and XNOR*

**Practice Drawing  
this Symbol!**



| A | B | $A \oplus B$ | $\overline{A \oplus B}$ |
|---|---|--------------|-------------------------|
| 0 | 0 | 0            | 1                       |
| 0 | 1 | 1            | 0                       |
| 1 | 0 | 1            | 0                       |
| 1 | 1 | 0            | 1                       |

# To Dos

---

Current assignment is due on Monday

Read handout (**Handout 5**) – must complete it by Monday!

Remember: **Lab on Friday**

</LECTURE>